Speech Script — Structure-Aware SpGEMM

APPT 2026 · Student GPU Programming Challenge · online talk (all English)

> **PRODUCTION NOTES — NOT SPOKEN**
>
> Pause at each "//" mark and at the end of every slide. Better a little slow than rushed.
>
> Editorial rules for this version: explain the reason, don't read the formula. Numbers survive only if the audience can do something with them; the exact thresholds stay on the slide and get pointed at, not recited. Plain spoken register — short sentences, concrete verbs, no "it is not X, it is Y" scaffolding.
>
> Spoken body = 1221 words → 9.4–9.8 minutes at 125–130 wpm with pauses. Inside the window — still rehearse against a timer. If it runs long on the day, the one remaining cut is slide 6's descriptor-cost sentence (~36 words); the Q&A covers it verbatim. Background the audience already has — the row-wise formulation, warp width — is deliberately not spoken; so is the symmetric-`A·Aᵀ` variant, which lives only in Q&A.

1 Title

Good afternoon, and thanks for having me. //

We were asked to multiply sparse matrices on a GPU — a hundred real matrices from SuiteSparse, two products each: A times A, and A times A-transpose. //

We built two things: a hash-based kernel of our own, and a small piece of logic that looks at a matrix and decides whether to run our kernel or hand the work to cuSPARSE. That second piece is what this talk is really about. //

One note on the setup. We compile for S-M eighty and target an A100, but we measured on an H100 with those same binaries. I'll come back to that.

2 Background

Both our products come out of the same row-wise merge: A times A, and the Gram product, A times A-transpose. //

Why is this hard on a GPU? Three reasons, and none of them is the arithmetic. **You don't know how big the answer is** until you have computed it, so you cannot allocate up front. **Row lengths are wildly uneven** — in one matrix, one row merges three rows and another merges thirty thousand. And **which memory you touch depends on the values**, so nothing prefetches. All three are the same thing, and the word for it is **irregularity**. //

And the challenge scores generality across the whole suite: brilliant on ten matrices and terrible on ninety scores badly.

3 The hard part

Here is what the suite looks like. A banded matrix from a P-D-E solver. A power-law graph, where a few rows hold most of the entries. A stiffness matrix, with dense little blocks down the diagonal. A circuit matrix with no symmetry at all. //

Tune a kernel for any one of these and you lose on the others. We measured that — on our kernel, and on the vendor's. //

Look at these three, all stiffness matrices. cuSPARSE picks a **pathological** code path here — one that goes badly wrong — and our very simple hash kernel beats it by ninety-six, a hundred and four, and **a hundred and ninety times**. //

So we stopped asking which kernel is fastest. The useful question turned out to be: **given this matrix, which kernel should run?**

4 Our engine

Our kernel hands each output row to one warp, which merges the incoming rows into a small hash table — all of its lanes writing at the same time. Every insert is an atomic compare-and-swap, so the result never depends on which lane gets there first. //

The interesting decision is **how big to make that table**. Here is the trick: before computing a row, we can already put a **ceiling** on it. Walk that row of A, add up the lengths of the rows you are about to merge, and the answer cannot have more entries than that sum. It is a loose ceiling — but it is free, and it is a ceiling, which is what we need. //

We allocate a third more than that, so the table stays at most three-quarters full and probes stay short, and we round up to a power of two so wrapping around is a bit-mask instead of a division. //

That same number decides where the table lives. Small enough for shared memory — about a thousand entries — and the row is accumulated entirely on-chip. Bigger, and it gets its own private slice of a scratch buffer in global memory. Short

rows get the fast path, long rows get a safe one, and no row can run over into its neighbour's. //

We walk each row **twice**. The first pass, the symbolic pass, only counts distinct columns, so we know exactly how big the output is before allocating a byte of it. The second does the arithmetic. //

Then one sort. We pack each entry's row and column into a single sixty-four-bit number, so sorting on that one key sorts by row first and column second — which is exactly CSR. One radix sort from CUB, scratch space allocated once and reused. //

For the Gram product we build the transpose explicitly and feed it to the same kernel, and that transpose is inside our timing. //

None of this uses anything Hopper-specific, and it matches cuSPARSE entry for entry on all two hundred and fifty-eight tasks.

5 The dispatcher

So neither implementation wins everywhere. Rather than pick one and live with it, we look at the matrix and choose. //

When a matrix loads we compute six numbers from its structure: how big it is, how many nonzeros, the average row length, **how uneven the row lengths are**, roughly how dense the output will be, and how symmetric the pattern is. All six come from one pass over the indices. We never run either kernel to find out. //

The rule is three comparisons deep — it is written out on the slide — and its shape is simple. Rows fairly even: use ours. Rows wildly uneven: use ours only when the matrix is big enough to be worth it and the output will not be too dense. Otherwise, cuSPARSE. //

Which of the six carries that decision? The one that measures **how uneven the rows are** — eighty percent of it, on its own. I like that, because row-length imbalance is what made this problem hard in the first place. **The feature that breaks the kernel is the feature that tells you which kernel to use.** //

One pass over the structure, three comparisons, and we profile nothing.

6 Results

Whole benchmark, two hundred and fifty-eight tasks, everything relative to cuSPARSE. //

cuSPARSE takes fifteen hundred and twenty milliseconds. Our kernel on everything takes one thousand and eighty-three — one point four times, but dragged down by the matrices it is bad at. The dispatcher takes **five hundred and seventy-nine**: **two point six three times**. //

The bottom bar is the **oracle**: four hundred and forty-six. So there were about a thousand and seventy milliseconds on the table, and we take nine hundred and forty of them. **Eighty-eight percent.** And all two hundred and fifty-eight results are still correct, entry for entry. //

Two caveats. Those six numbers cost under one percent on the small matrices, eight point six percent over the whole suite — almost all of it one host-side pass over our biggest matrices, which belongs on the GPU. And these are **H100** numbers — cuSPARSE uses its Hopper paths there and we use none, so on an A100 we would expect the gap to widen.

7 Robustness

In practice the dispatcher sent **two hundred and eleven** tasks to our kernel and **forty-seven** to cuSPARSE. //

On the small matrices, our kernel on its own is already **two point seven times** faster in aggregate, winning on about eight matrices in ten. //

Where it loses is the very large cases with dense output. One hash strategy is the wrong tool there; you want a merge-based accumulator, and we do not have one yet. Those are exactly the forty-seven the dispatcher hands off. //

That is what makes the system **robust by construction**, and it is the property I would like to leave you with: **we don't need our kernel to be the best kernel. We need to know when it isn't.**

8 Conclusion

The hard part of real-world SpGEMM is **generality**. //

We built a hash kernel that sizes itself per row and is correct everywhere, and a rule that reads six structural numbers and picks the right implementation without ever running a timing experiment. **Two point six three times** over the vendor library, driven by row-length imbalance. //

Route each matrix to whatever solves it fastest. //

Thank you — happy to take questions.

Handling Q&A — a few likely questions

> **"Why a third larger, and why a power of two?"**
>
> → We allocate `4/3` of the ceiling, so the table is at most seventy-five percent full and linear probing stays short without wasting shared memory. Rounding up to a power of two is what lets both the initial hash and the probe step be a bit-mask, `& (cap - 1)`, instead of a modulo.

> **"How do you handle collisions and races between lanes?"**
>
> → Linear probing on a multiplicative hash, with **every probe an `atomicCAS`**. The CAS hands back whatever was in the slot: `-1` means this lane just claimed it, our own key means someone else already inserted it, anything else means keep probing. No lock, and no assumption about lane ordering. We deliberately do *not* read the slot before the CAS — in shared memory that read can be cached and miss another lane's write, which duplicates keys across slots.

> **"The Gram product is symmetric — did you exploit that?"**
>
> → We did build that variant: hash only the upper triangle, then mirror. It halves the merging work, but you pay for a mirror pass and still sort the full result. Net it is a *loss* on small matrices — zero point nine two times — and a gain at scale: one point one two on medium, one point two six on large. It is byte-exact against the plain Gram product on all one hundred and twenty-nine matrices we ran. So the symmetry optimisation is itself size-dependent, which makes it a natural third option for the same dispatcher.

> **"Isn't the ceiling very loose on dense rows?"**
>
> → It can be, and that is the honest cost of the approach: we sometimes allocate a much larger table than the row needs. The saving is that we never have to grow, retry, or spill mid-row. On the very large high-fill matrices that looseness is part of why we lose — and part of why those go to cuSPARSE.

> **"Why not a merge-based accumulator?"**
>
> → Hashing is the right default for the small-to-medium regime that dominates the suite, and it keeps us at one kernel rather than a bin-sorted family. Merge-based accumulation is precisely what we are missing on the very large rows; adding it as a third branch under the same dispatcher is the next step.

> **"Why not use Tensor Cores?"**
>
> → The output structure is unknown and the accesses are scattered, so there is no dense tile to feed a Tensor Core. Like cuSPARSE and prior work such as nsparse and spECK, our accumulation runs on the CUDA cores. Tensor-core SpGEMM helps when the matrix has dense block structure, which the general SuiteSparse suite does not have.

> **"Is the dispatcher trained or learned?"**
>
> → No. It is a fixed depth-three tree distilled from the structural descriptors, which keeps it zero-profiling and fully portable. A learned cost model over the same six numbers is the natural next step.

> **"What does the dispatcher actually cost?"**
>
> → Under one percent on the small-matrix regime — zero point zero one to zero point one four milliseconds. Across the whole suite it is eight point six percent, and essentially all of that is one host-side `O(nnz)` pass over the largest matrices. Computing the descriptors on the GPU, or fusing them into the CSR load, removes it.

> **"These are H100 numbers — will they hold on A100?"**
>
> → The binary is S-M eighty only, so it runs identically on A100. cuSPARSE currently gets its faster Hopper paths on the H100 while we use none, so our relative speedups are, if anything, conservative. Reproducing them natively on A100 is first on our list.

> **"Did you test the official matrix set?"**
>
> → Our Group 1 is a *proxy*: the first hundred square real matrices in SuiteSparse's default order, which matches the official size regime. We added a second group of forty-one structurally diverse matrices — up to one point one million rows — specifically to stress generality.

> **"Did you exclude any matrices?"**
>
> → Only by an output-size guard, never by result. For A times A-transpose on power-law graphs the Gram output can become near-dense and exceed memory, so twelve of the forty-one diversity matrices are excluded by an upper bound on output size. That bound is structural, and applied before we know any timing.

> **"How do you guarantee correctness?"**
>
> → Byte-exact structural nonzero count, plus order-independent value checksums, against both cuSPARSE and a SciPy reference, on all two hundred and fifty-eight tasks — zero mismatches. We also cross-check the symmetric variant against the plain Gram product.

Pronunciation Guide (生词读音表)

Format: word — IPA — simple respelling (stress in CAPS). Practice the ones marked ⚑ out loud a few times.

Core technical terms

WORD	IPA	SAY IT LIKE
⚑ SpGEMM	/ ɛs piː dʒɛm /	"S-P-**gem**" (spell S-P, then "gem")
⚑ sparse	/ spɑːrs /	"sparss"
⚑ SuiteSparse	/ swiːt spɑːrs /	"**SWEET**-sparss" (*suite* = "sweet", never "suit")
matrix / matrices	/ ˈmeɪtrɪks / · / ˈmeɪtrɪsiːz /	"**MAY**-triks" / "**MAY**-tri-seez"
⚑ Gram (product)	/ græm /	"gramm"
transpose (noun)	/ ˈtrænspoʊz /	"**TRANS**-pohz" (as in "A-transpose")
symmetric / symmetry	/ sɪˈmɛtrɪk / · / ˈsɪmɪtri /	"si-**MET**-rik" / "**SIM**-i-tree"
⚑ irregularity	/ ɪˌrɛgjəˈlærɪti /	"ih-reg-yuh-**LAIR**-i-tee"
⚑ dispatcher	/ dɪˈspætʃər /	"dis-**PATCH**-er"
⚑ hindsight	/ ˈhaɪndsaɪt /	"**HYND**-site"
⚑ millisecond	/ ˈmɪlisɛkənd /	"**MIL**-i-sek-und"

Algorithm terms — rehearse these

WORD	IPA	SAY IT LIKE
⚑ ceiling	/ ˈsiːlɪŋ /	"**SEE**-ling"
⚑ probe / probing	/ proʊb /	"prohb" / "**PROH**-bing"
⚑ bit-mask	/ ˈbɪt mæsk /	"**BIT**-mask"
⚑ symbolic	/ sɪmˈbɑːlɪk /	"sim-**BOL**-ik"
⚑ radix (sort)	/ ˈreɪdɪks /	"**RAY**-diks"
⚑ scratch (buffer)	/ skrætʃ /	"skratch"
⚑ prefetch	/ ˌpriːˈfɛtʃ /	"pree-**FETCH**"
⚑ atomic (compare-and-swap)	/ əˈtɑːmɪk /	"uh-**TOM**-ik"
⚑ accumulator	/ əˈkjuːmjəleɪtər /	"uh-**KYOO**-myuh-lay-ter"

Kernel / hardware terms

WORD	IPA	SAY IT LIKE
⚑ cuSPARSE	/ kjuː spɑːrs /	"cue-**sparss**"
⚑ CUB	/ kʌb /	"cub" (like the animal)
warp / lane	/ wɔːrp / · / leɪn /	"worp" / "layn"
⚑ hash / hashing	/ hæʃ /	"hash"
⚑ nonzeros	/ nɑnˈzɪroʊz /	"non-**ZEER**-ohz"
stiffness (matrix)	/ ˈstɪfnəs /	"**STIFF**-ness"
vendor	/ ˈvɛndər /	"**VEN**-der"
⚑ Hopper	/ ˈhɑːpər /	"**HOP**-er"

Q&A only — not in the talk, but rehearse for questions

WORD	IPA	SAY IT LIKE
⚑ nsparse	/ ɛn spɑːrs /	"N-sparss"
⚑ spECK	/ spɛk /	"speck"
⚑ descriptor	/ dɪˈskrɪptər /	"dis-**KRIP**-ter"
⚑ multiplicative	/ ˌmʌltɪˈplɪkətɪv /	"mul-ti-**PLIK**-uh-tiv"
⚑ modulo	/ ˈmɑːdʒəloʊ /	"**MOJ**-uh-loh"

Easy to mis-stress (watch these)

WORD	CORRECT STRESS
generality → generality	jen-er-**AL**-i-tee
robust → robust	roh-**BUST** (stress 2nd)
an**al**ytics → analytics	an-uh-**LIT**-iks
in **agg**regate → in aggregate	"in **AG**-ri-gut"
irregularity → irregularity	ih-reg-yuh-**LAIR**-i-tee

Numbers you will say (rehearse these exact phrasings)

> **190× / 104× / 96×**
>
> → "a hundred and ninety times" / "a hundred and four times" / "ninety-six times"

> **258**
>
> → "two hundred and fifty-eight"

> **~1000 entries**
>
> → "about a thousand entries"

> **64-bit**
>
> → "a single sixty-four-bit number"

> **1520 ms**
>
> → "fifteen hundred and twenty milliseconds"

> **1083 ms**
>
> → "one thousand and eighty-three"

> **579 ms / 446 ms**
>
> → "five hundred and seventy-nine" / "four hundred and forty-six"

> **1070 / 940 ms**
>
> → "one thousand and seventy" / "nine hundred and forty"

> **2.63× / 1.40× / 2.7×**
>
> → "two point six three times" / "one point four times" / "two point seven times"

> **88% / 80% / 8.6%**
>
> → "eighty-eight percent" / "eighty percent" / "eight point six percent"

> **211 / 47**
>
> → "two hundred and eleven" / "forty-seven"

> **sm_80**
>
> → "S-M eighty"

> **A100 / H100**
>
> → "A one hundred" / "H one hundred"